

Basics of Object Detection

WHAT IS OBJECT DETECTION

≠ Image Classification!

↑
where we assign a label to a single image / frame



• What?

It involves identifying and locating multiple objects within an image or video

↗ here we assign also spatial information with a bounding box.

- What and
- Where?

coordinate-location of the object within the image



In Obj Detection

task:

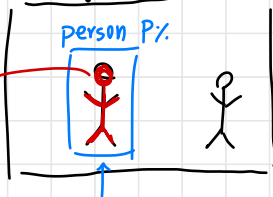
- Input: image / frame of a video or a Batch of images.

- Output:
 - ▶ Bounding Box $\equiv$ (location) coordinate of the object
 - ▶ Class Labels $\equiv$ Name of Detected Classes
 - ▶ confidence score $\equiv$ Probability (0,1) of the detected object

WHAT ARE BOUNDING BOXES?

Given an image

Image



Bounding Box = rectangle Box around Detected class

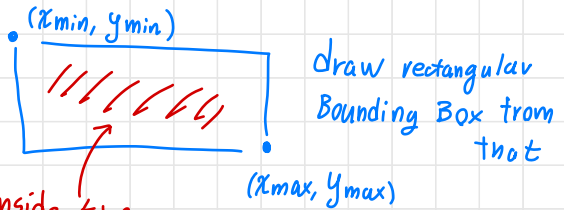
It defines the position of an object within the image
(+ multiple objects can be present)



How the Network will predict the coordinate? $\Rightarrow$ learnt during training

$\begin{cases} x_{\min}, y_{\min} : \text{Top left corner coordinates} \\ x_{\max}, y_{\max} : \text{Bottom right corner coordinates} \end{cases}$

$\downarrow$
Given this coordinates



coordinate system properly adapted to open cv



inside the Box there is the detected object

Data used for OBJECT DETECTION

- While in image classification we pass Images, Labels.

For Object detection Images, Labels

TRAINING DATA

- Images
- Annotation Files

Class Labels Bounding Box coordinates

class labels
Bounding Box (BB)

$\Leftarrow$ Required by object detection Pipelines

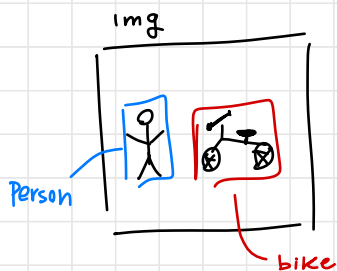
$\uparrow$
to generate

DATA ANNOTATION FILES many tools exists, e.g.

- Labelbox
- Roboflow
- Supervisely

$\hookrightarrow$ and the file has a specific template

e.g. With Makesense tool for Detection annotation
(containing open source datasets)



you define the present Labels
in the image

↪ you can here define the BB with
associated classes

↓

it can be exported in different
file format

e.g. in the YOLO format: ↪ Annotation File

row: class label xmin ymin xmax ymax

(there are different formats...)

↑

help during training the network

↑ in normalized
image coordinates

OBJECT DETECTION METRICS

- Different competitions introduce different metrics



PASCAL VOC : (2007/2012) → different formats for Benchmarking

COCO DETECTION : 80 classes, one of most used

ImageNet

...

- Definitions

Intersection Over Union (IoU) measure the overlap between Bounding Boxes

↳ It requires the GROUND TRUTH and the predicted (from model)

$\begin{cases} \text{Ground Truth: } B_{gt} \\ \text{Predicted: } B_p \end{cases}$

↳ IoU evaluate the quality as overlapping area divided by union

$$IoU = \frac{\text{area}(B_p \cap B_{gt})}{\text{area}(B_p \cup B_{gt})}$$

+ Other machine learning / computer vision:

- True Positive (TP) = correct detection $IoU > \text{threshold}$ Detection
- False Positive (FP) = Wrong detection $IoU < \text{threshold}$ Detection
- False Negative (FN) = Not detected yet
- ~~True Negative~~ correct misclassification NOT useful in detection

based on use cases
(50%, 75%, 95%)
hyperparameter



PRECISION = $\frac{TP}{TP+FP}$ ability of model to identify only relevant objects. $\equiv$ percentage of correct pos prediction
all detections among all detections

RECALL = $\frac{TP}{TP+FN}$ ability of model to find all relevant objects.
percentage of TP among all relevant or
all ground truths

Given the basic values:

Metrics:

1) Precision x Recall curve

2) Average Precision: different methods

↗ 11-points interpolation

↘ All-points interpolation

• 11 Points Interpolation

Tries to summarize shape of PR curve averaging the precision at a set of eleven equally spaced Recall levels

$$AP = \frac{1}{11} \sum_{r \in \{0, 0.1, \dots, 1\}} P_{\text{interp}}(r)$$

• All points AP = ...

→ given a set of N_{img} images each img with N_p predictions,

defined the threshold of the IoU, we define $N_{\text{TP}}, N_{\text{FP}}, N_{\text{FN}}$

and we can easily define of TP, FP, FN in an image

↪ rearranging the data in TP, FP relevant

Counting cumulated (Acc) TP and FP after reordering the Detection based on the confidence score

(Where $\text{Recall} = \text{TP} / \text{All Ground Truth} = (\text{TP} + \text{FN})$)

↳ you can reconstruct the Precision-Recall curve from here



↪ on top of this you apply the 11-points interpolation OR the complete one (using the next 11 points)

↓
to compute Average Precision: AP

+ All interpolated points can be used to compute AP (Average)

↑
Using the Precision-Recall curve. → using an average area

↑
The **Average Precision** is the main metric!

↳ When using more labels...

it came the concept of Mean Average Precision **mAP**

(With All points interpolation as the most used computation method)

BOUNDING BOX

Rectangular Region in an Image enclosing an object detected by an OBJECT DETECTION MODEL

↳ helps in locating + identifying objects in an image providing spatial coordinates.

↑ class + position



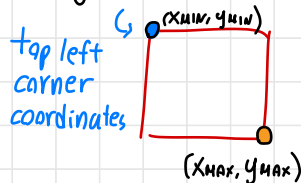
There are different representations for BB

1) Corner Format

2) Center Format

1) **CORNER FORMAT:**

using $(x_{min}, x_{max}, y_{min}, y_{max})$ Representation as 4 components vector



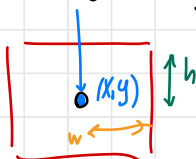
bottom right corner

used around image of the object class



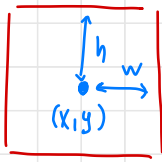
2) **CENTER FORMAT:**

using $(x_{center}, y_{center}, \text{Width}, \text{height})$ Representation



dimension of the BB in pixelspace

using (x_c, y_c, w, h) you can fully reconstruct the BOUNDING BOX



h, w as half
dimensions of
the BB

→ The CENTER POINT
 (x_c, y_c) allow to compute the rest
eg. $\begin{cases} x_c = 125, y_c = 105 \\ w = 150, h = 150 \end{cases}$

- Different Algorithms : Different types of Obj Detection Alg
uses the BB Formats : CENTER / CORNER

$\begin{cases} \text{CENTER) Majorly used!} \\ \text{CORNER) Not A lot today (2025): previously used a lot!} \end{cases}$
↑

In both cases you need 4 informations to fully define it in 2D

using YOLO → coding with Python Notebook...

with pre-trained models + comparing the results

Next addressing custom model training

↓

using ULTRALYTICS YOLO 11 ⇒ v11 as the last version of 2025
(many documentations on official website)

(Which can be used also for segmentation, tracking, etc...)

different models for lightweight, performance, mode....

↓
nano, small, medium, large, xtra large
n s m l x

↑

possible problems with overlapping objects in images
with respect to FastRCNN

Using **Detectron 2** (facebook AI research)

↓

SOTA detection + segmentation algorithms

(refer to official github repo to use the models and a demo .ipynb)

e.g. **Model zoo** = collection of many models ready to be used
With detectron2

↓

you use the Default Predictor to perform prediction
+ additional classes to handle predictions - training - visualization

relying on configuration files downloadable from GitHub repo.

e.g. **MaskRCNN** OR **Detection Networks** can be selected
using **pkl** configuration file of the model weights.

OBJECT DETECTION ARCHITECTURES

to Detect objects in the Image. The Area of the localized object needs to be detected with good Accuracy, Speed, Robustness

Based on the objective different architectures exists:

- 1) Two-stage object detectors (Region Proposal)
- 2) One-stage object detectors (Regression)
- 3) Transformer-based object detector (Recent Advances)

↳ Main tasks of an OBJECT DETECTOR

CLASSIFICATION + LOCALIZATION

Identify WHAT
(class)

determine WHERE
(Bounding Box)

1) Two-stage (Region Proposal)

1.1) RCNN [2014] Region Proposal (CNN) Network
using CLASSICAL COMPUTER VISION Approaches

Idea is to ← (such as SELECTIVE SEARCH)

select multiple

regions, Wrap it and pass them through a CNN (Very slow)

1.2) FAST RCNN [2015] (Girshick Author)

on top of RCNN to make them FAST!

Thanks to ROI pooling concept

↑
extract features and share convolution feature map.

again selective search is crucial

↑ try to replace it with Deep Learning ??

The Region proposal is not good

1.3) FASTER RCNN [2015] still used sometimes..



made faster using RPN (Region Proposal Network)

NOT Real time still...

↑
using deep learning approach
for Region Proposal
replace Selective Search!

1.4) Mask RCNN [2017]

Widely used for segmentation task!

(Also for Classification + Localization + Segmentation)

still Object Detection takes time...

↓ Two stage detectors are VERY ACCURATE but slow!

2) One stage detectors (Regression)

2.1) YOLO : You Only Look Once [2016]



NOW a family of architectures.. v1, v2... v11, v26

"One-stage" eliminate the Region proposal set up



predict class + BB directly ⇒ Much FASTER than two stage!

2016... multiple versions have been launched

2.2) SSD : Single Shot Multi-Box Detector [2016]

very used previously

Faster than FASTER RCNN + Handle small objects.

BUT less accurate than FASTER RCNN

2.3) Retina Net [2017]

uses FPN (Feature Pyramid Network)
with Multi-scale detection ↓

High accuracy + Real time speed! ← you can use it in Real-time
BUT computationally MORE EXPENSIVE than YOLO

(balance Precision, Recall better than YOLO, SSD)

3) Transformed-based architectures New trend!

3.1) DETR (DEtection TRansformer) [2020] Facebook-AI Group
COMPUTATIONALLY EXPENSIVE + Not Real time (expensive)

- Find objects relationship using Attention mechanism
+ Eliminates NMS (Non Max Suppression)

- End-to-End object detection without ANCHORS!
(E2E) without anchors
↑
Replaced by TRANSFORMERS
- concept used in the others detector as pre-defined hyperparameter

3.2) Deformable DETR [2021]

Deformable attentions focusing on Relevant Regions → making DETR more efficient
↓
to focus only on relevant objects

3.3) DINO : (DETR with Improved Noisy Optimization)

↓
It improves training convergence and accuracy of detection

In summary...

Comparison of Object Detection Architectures

Architecture	Type	Speed	Accuracy	Notable Features
R-CNN	Two-Stage	Slow	High	Uses Selective Search
Fast R-CNN	Two-Stage	Faster	High	RoI Pooling
Faster R-CNN	Two-Stage	Faster	Very High	RPN for region proposals
Mask R-CNN	Two-Stage	Slower	Very High	Adds segmentation masks
YOLOv3	One-Stage	Very Fast	High	Multi-scale detection
YOLOv5	One-Stage	Real-time	High	Lightweight, efficient
SSD	One-Stage	Fast	Medium	Multi-scale feature maps
RetinaNet	One-Stage	Moderate	Very High	Focal Loss for imbalance
DETR	Transformer	Slow	Very High	End-to-end detection
Deformable DETR	Transformer	Faster	Very High	Focused attention mechanism



Detailed description of How the main architectures work...

RCNN : Regions with CNNs

deep Learning Obj Det. framework by Ross Girshick in 2014



one of the first approaches

[R-CNN: Regions with CNN features]

Successfully working for OBJ DET. with Deep Learning

Previous approaches to Object Detection: (based on classical cv)

- HOG: Histogram Oriented Gradient
- SIFT: Scale Invariant Feature Transform
- Selective Search:

^
NOT accurate, NOT efficient! → specifically used for obj Detect

In R-CNN the deep learning concept is introduced
+ classical Selective Search technique

Tasks: Localization + Classification
 (Regression) (Classification)
 ↓ ↓
 finding the coordinates finding the class

the idea is to perform CNN based features extraction using the classifier (CNN) while the area is selected by a REGION PROPOSAL technique

⇓ ^{What are the IMPORTANT objects/areas within an image?}

1) Region Proposal methods:

- o Selective search ^(potential) Generate candidate locations ^{possibilities that an object is there}

↓
Using classical

CV approaches, find if there can be an object or not.

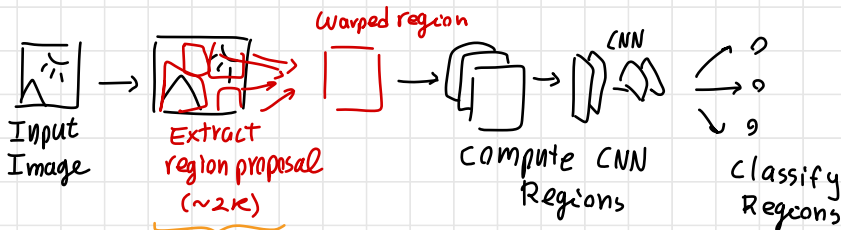
↑

the network proposes many candidate object location $\sim 2k$
 shared by Region proposal

↓

2) then CNN extract features: CNN Feature Extraction

↪ 2 step ! ↪ PIPELINE of RCNN
 entire architecture steps.



This is the Bottleneck of the Architecture

↑

(by Selective search)

1) Region Proposal → proposing candidate locations
(extract regions)
• selective search ~2K potential locations
(rectangular BB that may contain objects)

2) Feature Extraction → using image classification architecture
by CNN (e.g. VGG, AlexNet...)
(extract feature vector)
• SOTA Classification Architecture
all Region proposal are converted to expected image size
input of the CNN Architecture! (Resize operation)

(classify from features)
3) Classification task
↓
you classify extracted features by CNN
• SVM was used
Support Vector Machine
for classification
↙ on the extracted features from the image

4) Bounding Box Regression → BBox fine tuned coordinates
↓
final Results: Class + Location

LIMITATIONS:

- SLOW INFERRING (high prediction time)
* computationally Expensive
(because of the Proposed Regions ~2K)
(takes lot of time)

on GPU ~ 50 sec per Image (extremely SLOW)
↘ ~ 2 min in CPU

How to reduce the # Regions? FIRST Required impr.

- STORAGE OVERHEAD: feature extr. & region.. leading to high storage requirements on GPU

• NOT End-to-End Trainable

The entire architecture CANNOT be trained!



It is broken in multiple components

- 1) Region Proposal
- 2) CNN features extraction
- 3) SVM Classifier

} each component is trained separately!!

↑
BIG LIMITATION

make pipeline inefficiently

IMPROVEMENTS...

1) FAST RCNN

2) FASTER RCNN (e.g. in Detectron2 models)

In practice RCNN came out in 2014, nowadays it is DEPRECATED but fundamental for next architecture

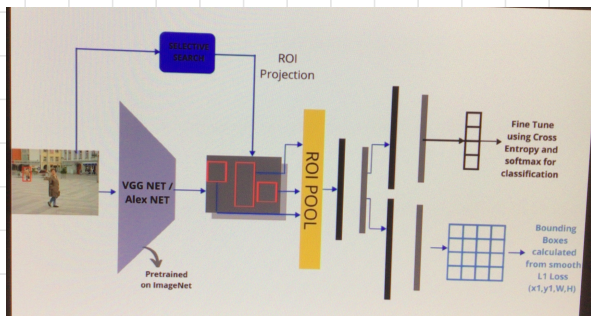
FAST-RCNN

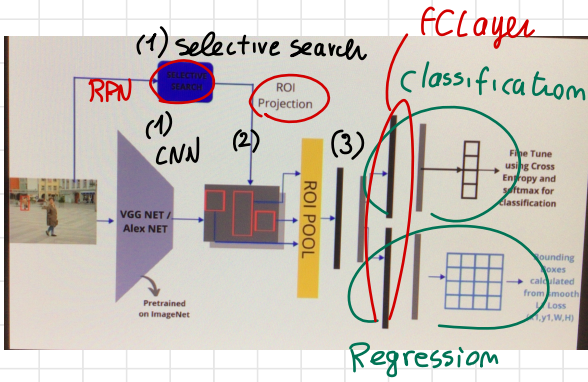
Object Detection With Bounding Box Refinement

While R-CNN performs separate Forward passes & region proposal, Fast-RCNN speeds up it with a SHARED FEATURE MAP

□ Classifier and Localization run in PARALLEL
BB Regression

} done SIMULTANEOUSLY
in FAST R-CNN





PIPELINE

1) Input Image processing
Instead of using selective search

The image is SIMULTANEOUSLY passing CNN + selective search module

Input Image → CNN
→ Region Proposal

It removes the R-CNN redundancy of processing each region separately.

in R-CNN each region is being processed by CNN model

⇒ REDUNDANT procedure!!

↓
Here in FAST R-CNN directly CNN on input image

2) Region Proposal Extraction → Location...
Instead of an external selective search

↑
The Region Proposal are taken from RPN
→ providing ROI: candidate object locations

• NO external selective search → take it from RPN directly

3) ROI pooling: extracted feature maps are used to
↑ generate ROI features.

It uses the extracted feature maps previously by CNN.
Generate from ROI features

Know exactly where is the object? ← using ROI projection

ROI pooling layer rely on RPN region proposal + CNN features

ensures that all region proposal are converted into fixed size before classification and regression

↳ Reshaping operation → Fixed size region proposal
↓ feature vector as pooled features:

4) Fully connected layer after ROI Pool
* using POOLED FEATURES. passed directly

5) Simultaneous CLASSIFICATION + BOUNDING BOX Regression

While RCNN rely

on SVM for classification ⇒

here softmax based classification
is used.

refine BBox
location

Advantages (wrt RCNN)

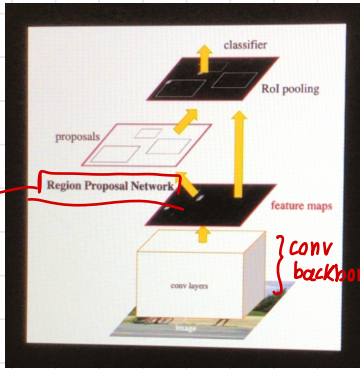
- Improve speed significantly! (While RCNN was using 2K Regions)
- End-to-End training: the entire architecture is trained in ONE SHOT!
- Better accuracy than RCNN and SPPNet
- Memory efficient (∀ region proposal you don't extract features as RCNN... But just on the full image)

Disadvantages

- Selective search module (still NOT Deep learning based) is a traditional CV technique, slow + NOT General
- NOT Real-Time: high inference time (~20s on GPU)
- NO Region proposal learning → We use selective search, NO learning is applied to select regions.. they are FIXED and NOT learned

FASTER RCNN (still used today architecture for Obj Detection) and Img segmentation (Mask RCNN)

Advanced model that introduce a Region proposal network to improve the previous models based on selective search



CNN Network

ARCHITECTURE

1) convolution backbone

: responsible for feature extraction



↓
= CNN (ResNet4, VGG..) (conv. backbone)
↓
feature map



2) Region Proposal Network (RPN)

fully convolution NN, Replacing selective search

↳ scan feature maps to search for Region Proposal

Fully conv network: purpose of generate Region proposal

- RPN predict if a region contains an object or not, refining the Box after that

- Using ANCHOR BOXES for the Region proposal: to detect objects at multiple scales.

3) ROI Pooling layer → resize region proposal in a fixed size feature representation

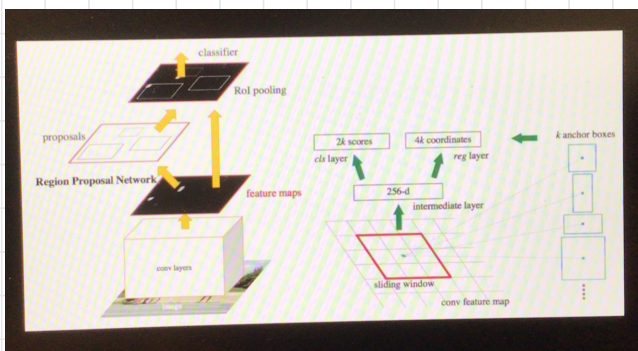
Resize Region Proposal (as done by RCNN)

Region proposal RPN → mapped into FEATURE MAP ←

ROI pooling resize proposals on top of feature maps

⇒ then finally you can use the feature map to classify.

↳ by resizing the model can adapt to MULTI-SCALE boxes



object to detect can be.

- small
- large
- thin
- ...

multiple scales are possible

different proposals boxes

4) Fully Connected layers: rely on fixed shape features, process by FC layer...

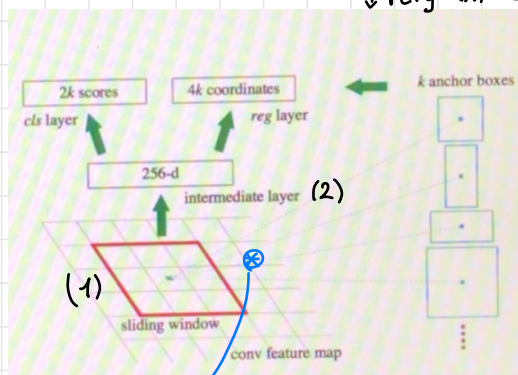
where 2 branches run

Classification
Object Class

Regression
BBX Coordinates

RPN Mechanism

↓ rely on a sliding window (1)



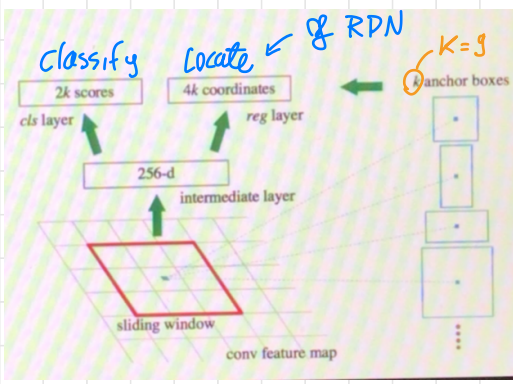
predicting Region proposals

on top of conv map you move the window (slide) searching for different objects.

Often small window (e.g. 3x3) to PREDICT object regions

↓

(2) then goes to an intermediate layer 256 dimensional feature space
↓
from sliding window information extracted become 256 dim



1. Sliding Window = predic region proposal
↓ pass to intermediate

2. 256 dim intermediate layer

3. then in parallel

a CLASSIFIER and REGRESSOR

INTERNAL TO RPN, to the
one of the overall model

Giving 2k scores output from
classification with $K := \# \text{anchors}$
 $2K = 2 * 9 = 18 \text{ scores}$

Foreground = $\left\{ \begin{array}{l} \text{Classifier decide if} \\ \text{there is an object or NO} \\ \text{(classify Region)} \end{array} \right\} \leftarrow \left\{ \begin{array}{l} \text{IF background or} \\ \text{NOT (No object)} \end{array} \right\}$

The Regression 4K coord.

IF there is an object

$4K = 4 * 9 = 36 \text{ coordinate}$

To refine the Boud Box coordinates
+ Adjust Anchor to better fix
objects

WHAT ARE exactly
ANCHOR BOXES?

↑
Sliding Window with $\Rightarrow$ scale + aspect ratio

• Sliding Window
+ take care of

• scale

• Aspect Ratio

} Relevant in every object

3 scales + 3 Aspect Ratios

are used in ORIGINAL FasterRCNN paper $\Rightarrow 3 \times 3 = 9 \text{ Anchors}$

+ Ex in a conv Feature map. $(W \times H) \sim 2400$, (W, H, K)
 $K := \text{anchors}$

↑ using multiple sizes of Anchor Boxes to understand + get region proposal
for classification layer...

The RPN is a CNN on its own, with its Classifier and Regressor (Network in Network)

Advantages

- high detection accuracy
- trainable End-to-End (or you can train RPN separately)
- Efficient RPN (replacing selective search)
- Handle multiple object scales

Disadvantages

- Computationally Expensive
- NOT Real time
- Sensitive to SMALL OBJECTS
↓
May NOT be detected!

- example Pytorch application: Using RESNET50 FASTER RCNN for vision model
↓
pre-trained on coco Dataset (91 classes)

still have some False Positive (FP) in Redetection of objects with smaller confidence score...

↳ control the detection threshold to make the FP rate decrease and be more PRECISE

YOLOv11 CUSTOM TRAINING

It is possible to TRAIN the YOLO model on a CUSTOM Dataset using ultralytics pythom library

↓
after training it, is it possible to use the pt model pytorch model
OR to extract the model for OPTIMAL INFERRING

↓
export using command line OR using Pythom API

e.g. { ONNX / OpenVINO CPU
OR TensorRT GPU

everything can be done with Pythomic Approach OR Command Line Interface
through Pythom code through Terminal commands

Detectron 2 (CUSTOM OBJECT DETECTION)

Training on CUSTOM DATASET → using Detectron 2 as model.



using pre-trained model + fine tuned on a different dataset

When DOWNLOADING the Dataset from RoboFlow: for Detection you need the COCO Format

JSON annotation file



e.g. an initial Detectron2 model is used, configuring the FASTER RCNN training parameters